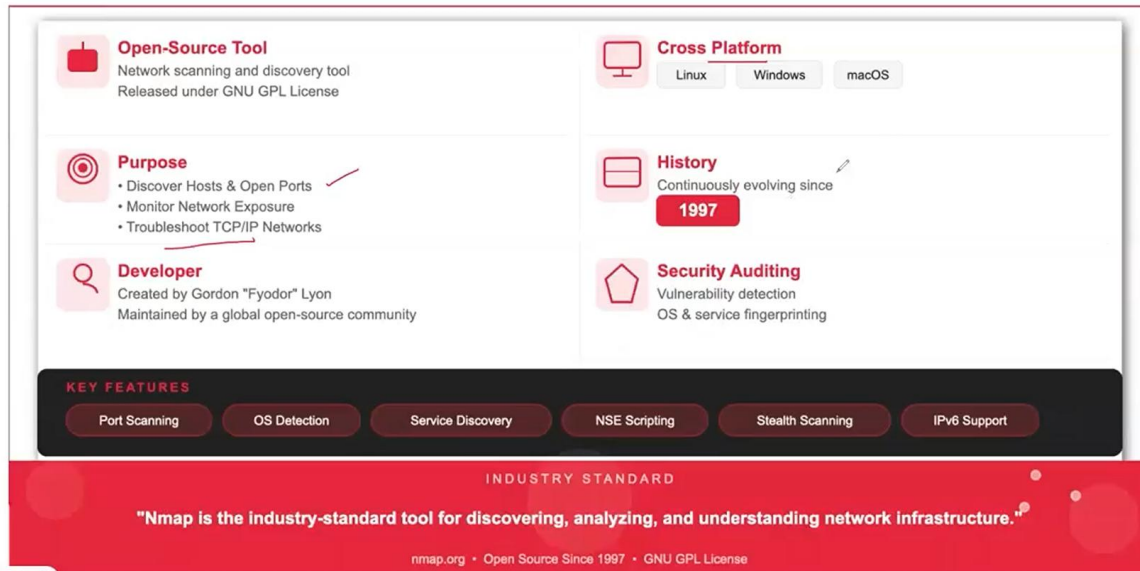


NMAP BOOTCAMP NOTES

INTRODUCTION TO NMAP

The world's most Trusted Open Source Network discovery & Security Auditing tool



Open-Source Tool
Network scanning and discovery tool
Released under GNU GPL License

Cross Platform
Linux Windows macOS

Purpose
• Discover Hosts & Open Ports ✓
• Monitor Network Exposure
• Troubleshoot TCP/IP Networks

History
Continuously evolving since
1997

Developer
Created by Gordon "Fyodor" Lyon
Maintained by a global open-source community

Security Auditing
Vulnerability detection
OS & service fingerprinting

KEY FEATURES

- Port Scanning
- OS Detection
- Service Discovery
- NSE Scripting
- Stealth Scanning
- IPv6 Support

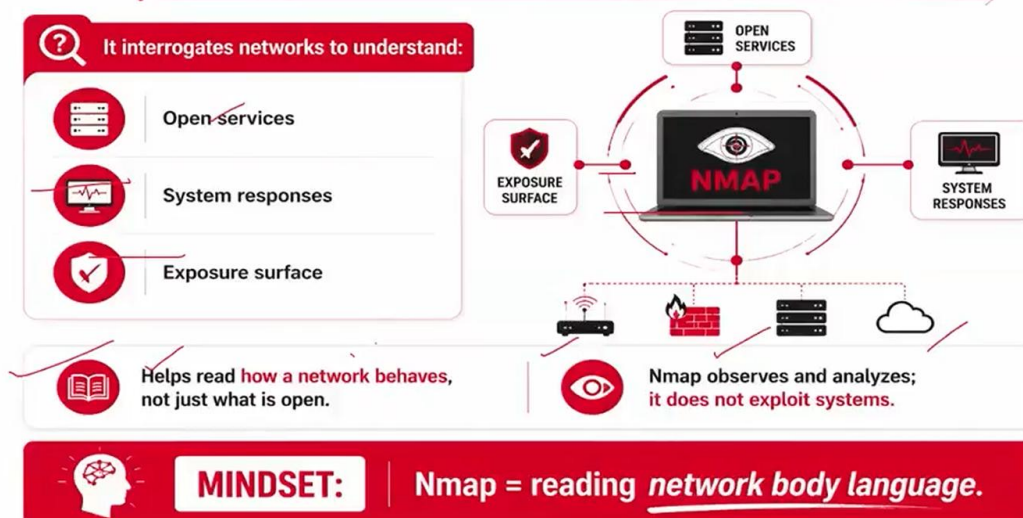
INDUSTRY STANDARD

"Nmap is the industry-standard tool for discovering, analyzing, and understanding network infrastructure."

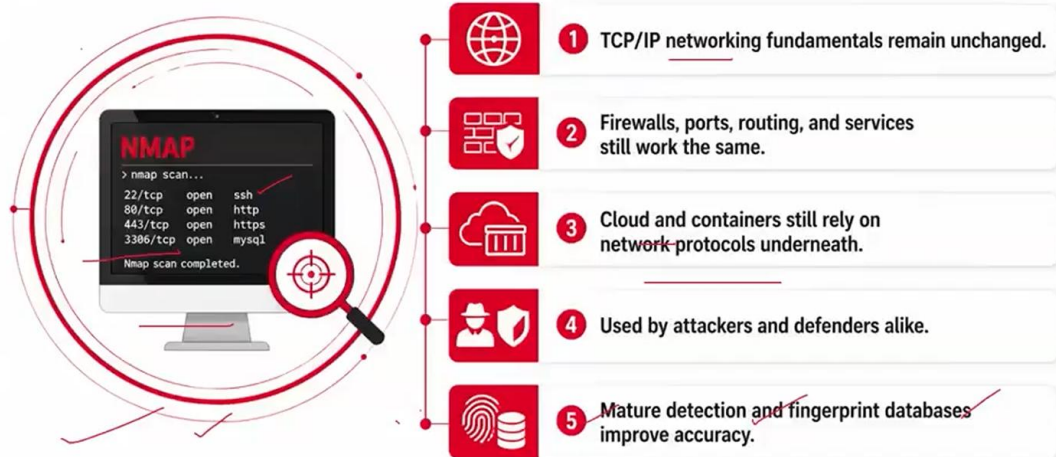
nmap.org • Open Source Since 1997 • GNU GPL License

WHAT NMAP REALLY IS ?

Nmap ≠ Only Port Scanner.



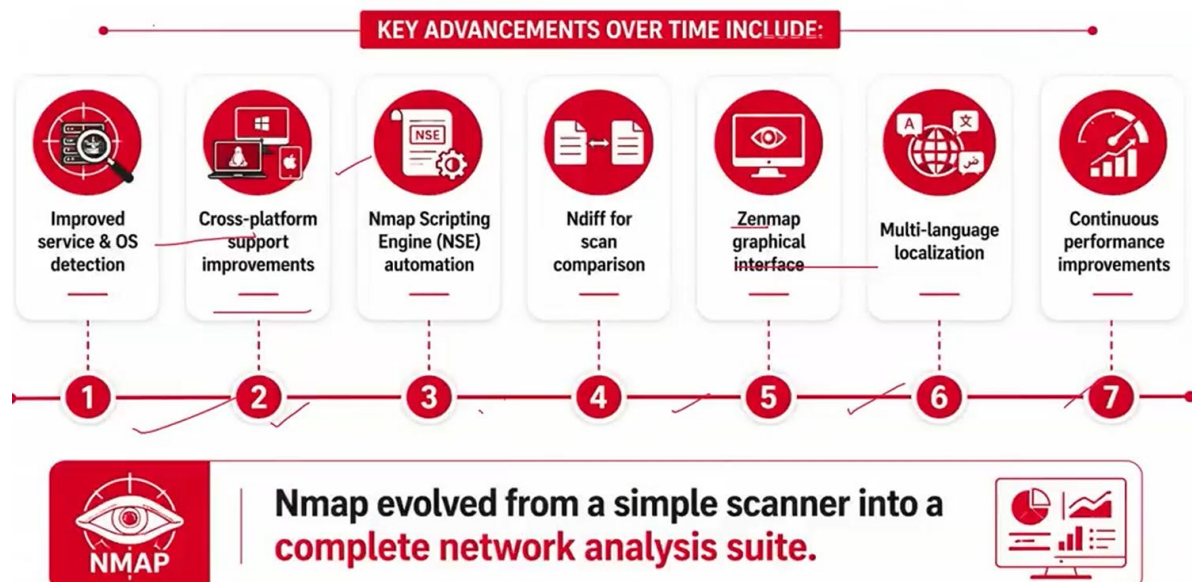
WHY NMAP STILL DOMINATES IN 2021



REALITY:

Technology evolves, but network fundamentals don't.

EVOLUTION OF NMAP CAPABILITIES



WHERE NMAP FITS IN CYBERSECURITY



LEGAL AND ETHICAL BOUNDARIES



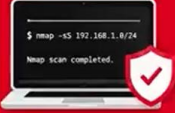
SCANNING ≠ EXPLOITATION



 Nmap performs **reconnaissance**, not intrusion. 

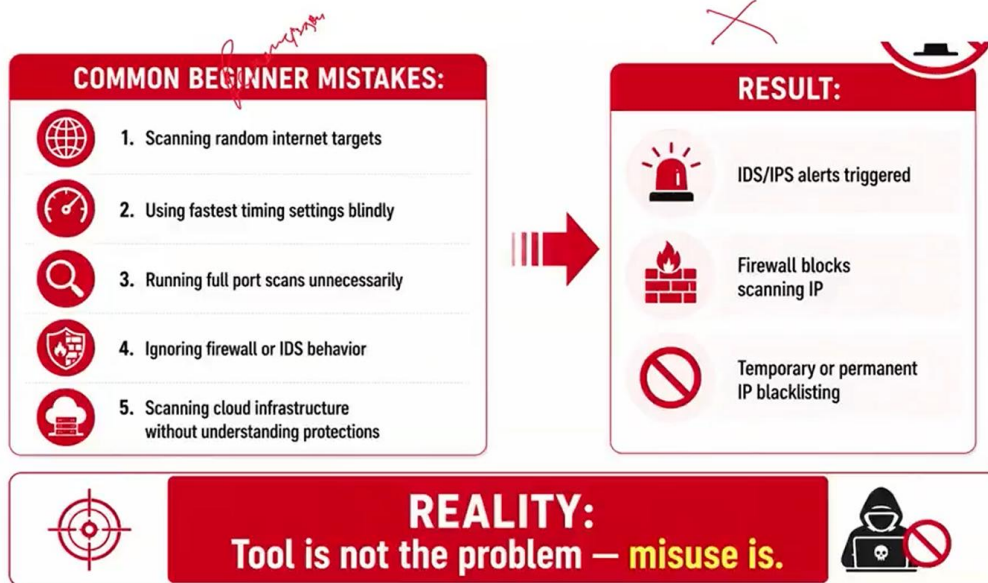


NMAP MYTHS , MISTAKES & Pro Mindset

 	Myth: <i>Perception</i> “Nmap is Illegal”	Reality - Nmap itself is a legal diagnostic tool. - Illegality depends on how and where it is used. - Unauthorized scanning may violate laws or company policies. - Same as any technical tool misuse creates legal issues.	Note: Always scan within permission or defined scope. Golden Rule: Authorization decides legality, not the command.
 	Myth: “Nmap is Noisy”	Half Truth - Aggressive scans create detectable traffic. - Properly tuned scans reduce detection chances. - Scan timing, probe type, and targeting decide noise level. - Noise is usually due to poor configuration, not the tool itself.	Key Insight: Bad usage is noisy; controlled usage is calculated. 

 Nmap is powerful. Your **mindset** and **method** make the difference. 

WHY BEGINNERS GET BLOCKED OR BANNED



WHEN NOT TO USE NMAP

AVOID USING NMAP WHEN:

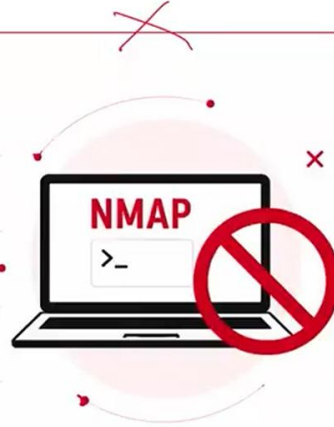
**1** No authorization is available

**2** Target network is **highly sensitive** production infrastructure

**3** You need **ultra-fast** internet-wide scanning

**4** Exploitation stage **already started**

**5** **Passive monitoring tools** are more suitable



**NMAP FOCUSES ON PRECISION, NOT SPEED.**

HOW NMAP ACTUALLY WORKS

 Nmap works by sending specially crafted packets and analyzing responses to learn about the target.

PACKET CRAFTING & RAW SOCKETS

Normally, operating systems handle network packets automatically when applications communicate.

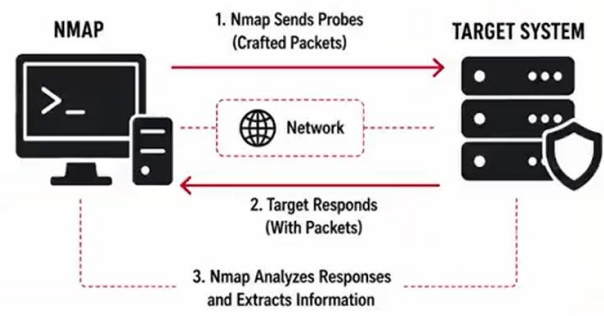
Nmap works differently:

-  It creates custom packets manually
-  Sends probes designed to trigger responses
-  Observes how the target reacts

This is called **packet crafting**.

To do this, Nmap uses **raw sockets**, which allow direct packet control instead of normal application networking.



HOW NMAP COMMUNICATES (AT NETWORK LEVEL)

```
graph LR; Nmap[NMAP] -- "1. Nmap Sends Probes (Crafted Packets)" --> Network((Network)); Network -- "2. Target Responds (With Packets)" --> Target[TARGET SYSTEM]; Target -- "3. Nmap Analyzes Responses and Extracts Information" --> Nmap;
```



INSTRUCTOR INSIGHT:
Nmap doesn't just talk to systems— it **carefully designs packets** to extract information.



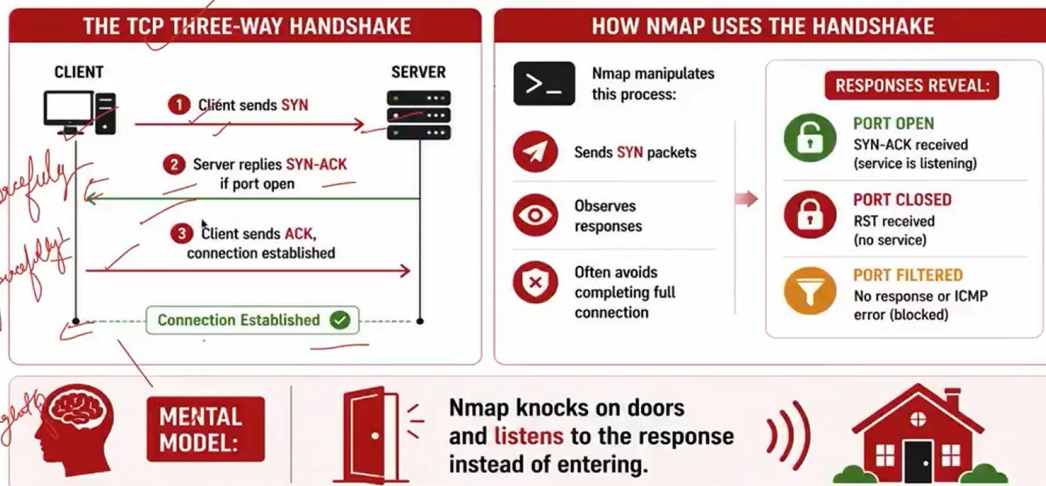
TCP COMMUNICATION FLAGS

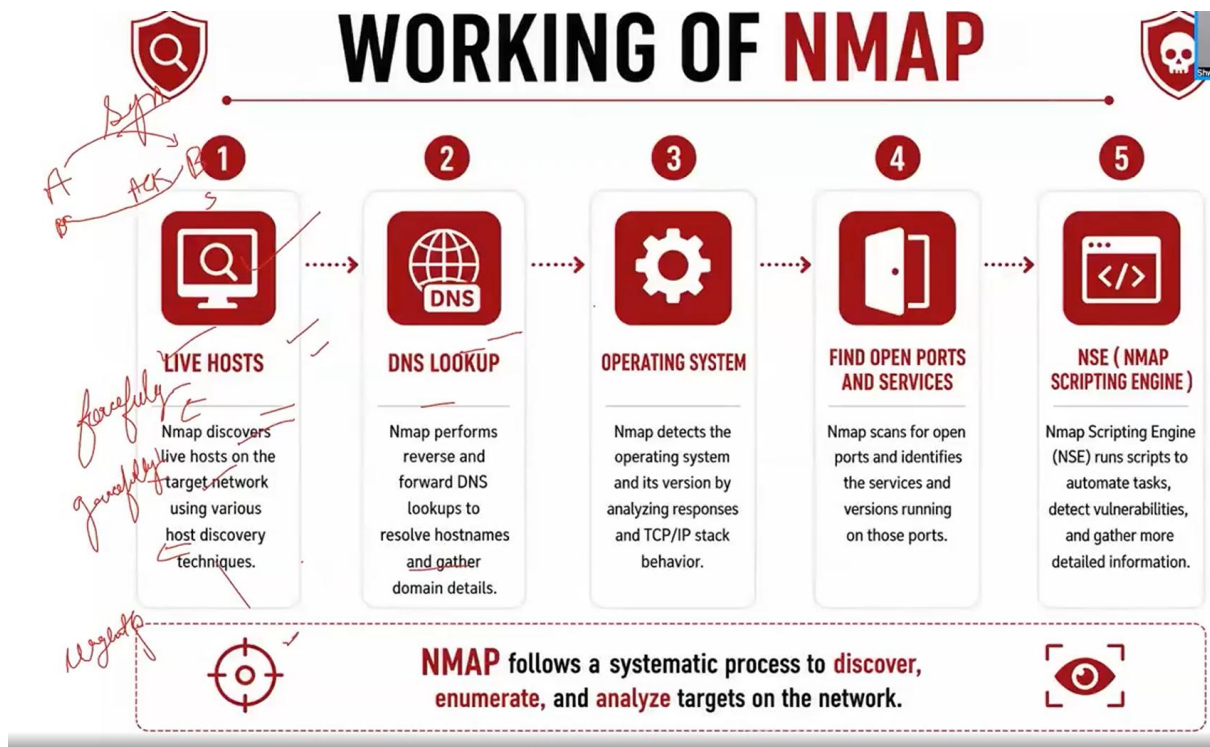
SYN (Synchronize)	 Synchronizes sequence numbers to initiate a connection.	 Purpose: Used at the beginning of a TCP connection (3-way handshake).	Example: Client wants to connect to server. 
ACK (Acknowledgment)	 Acknowledges receipt of data successfully.	 Purpose: Confirms that data has been received without errors.	Example: Receiver confirms data was received. 
RST (Reset)	 Resets the connection immediately.	 Purpose: Terminates a connection due to an error or issue.	Example: Connection is aborted due to error. 
FIN (Finish)	 Indicates sender has no more data to send.	 Purpose: Closes the connection gracefully.	Example: Sender finished sending data. 
PSH (Push)	 Pushes data to the receiver immediately.	 Purpose: Ensures data is delivered to the application without delay.	Example: Real-time data like chat or streaming. 
URG (Urgent)	 Marks data as urgent within the stream.	 Purpose: Signals that the data is urgent and should be prioritized.	Example: Used in rare urgent notifications. 
 TCP FLAGS control connection setup, data transfer, and termination for reliable communication . 			

TCP HANDSHAKE LOGIC

A
B
S
ACK

Most network services use TCP, which follows a **three-step handshake**:





Login to Kali Linux:

```
└─(root@LAPTOP-RREKKFPQ)-[/mnt/c/Users/Janya]
```

```
└─# ifconfig
```

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1480
```

```
inet 172.30.67.34 netmask 255.255.240.0 broadcast 172.30.79.255
```

```
inet6 fe80::215:5dff:fe91:856a prefixlen 64 scopeid 0x20<link>
```

```
ether 00:15:5d:91:85:6a txqueuelen 1000 (Ethernet)
```

```
RX packets 87726 bytes 331151771 (315.8 MiB)
```

```
RX errors 0 dropped 0 overruns 0 frame 0
```

```
TX packets 18851 bytes 1750193 (1.6 MiB)
```

```
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
```

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```























































































































































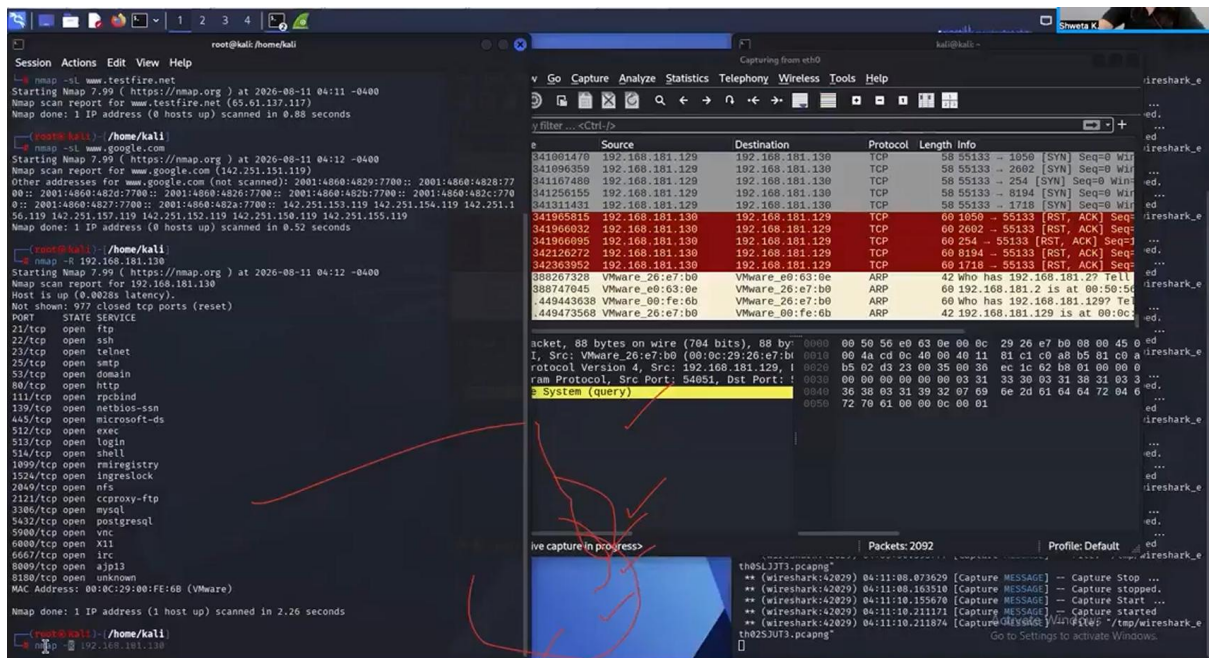



```



REVERSE DNS LOOKUP

REVERSE DNS LOOKUP



OPERATION SYSTEM

ACTIVE BANNER GRABBING

PASSIVE BANNER GRABBING

Windows TTL 128

Linux TTL is 64

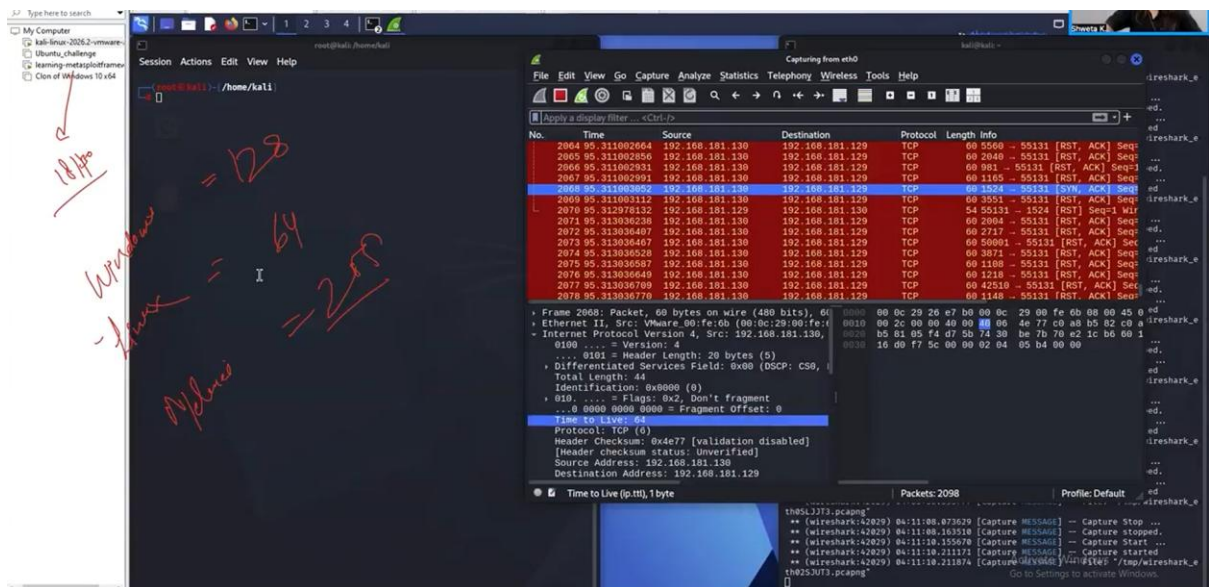
Network device TTL is 2

Passive banner grabbing from ubuntu – OS detection

Version of the OS

CPE value

Kernel value

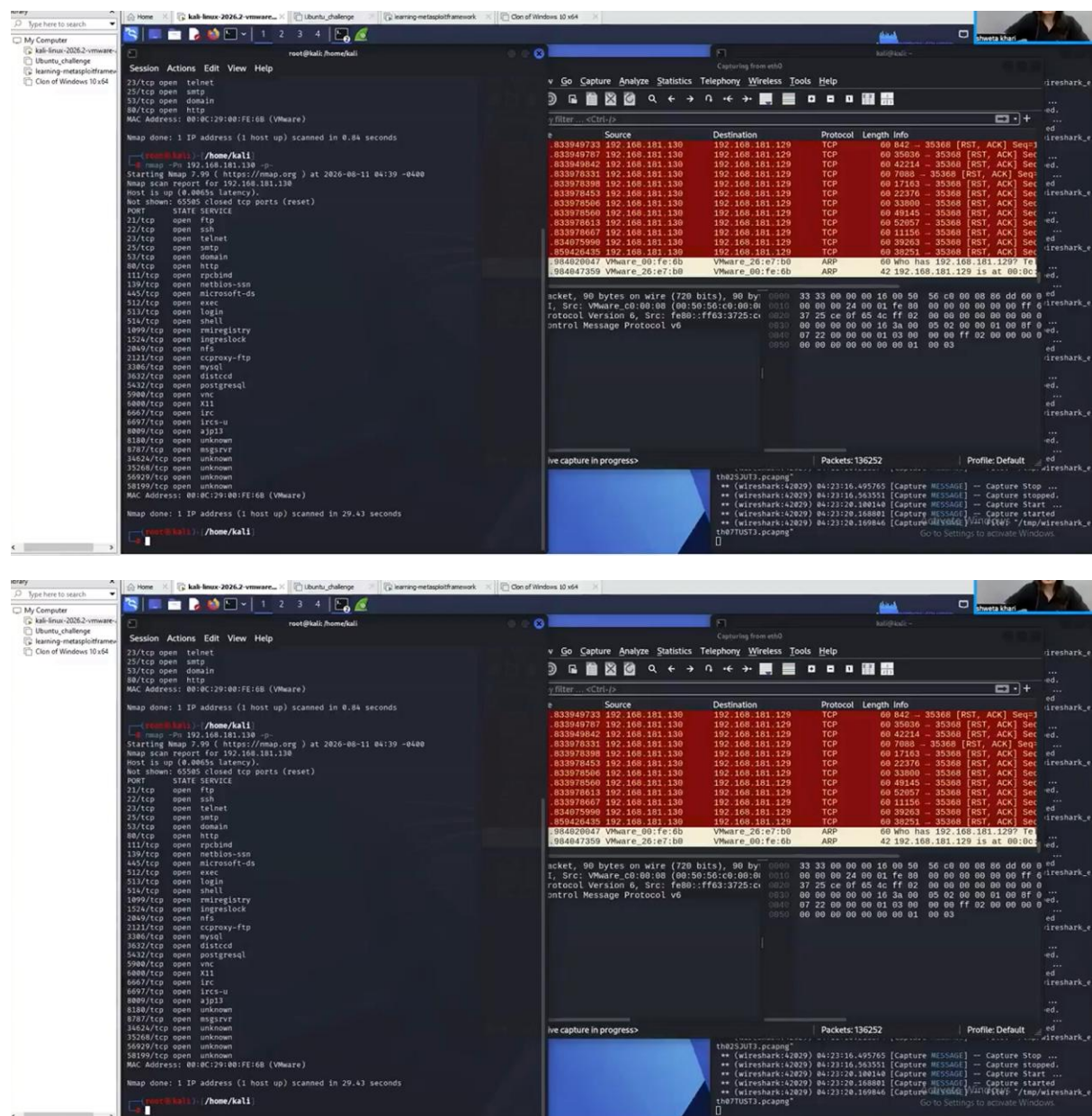


Perform only initial 10k port out of the 65,535

\$\$nmap -Pn <IP> -p -

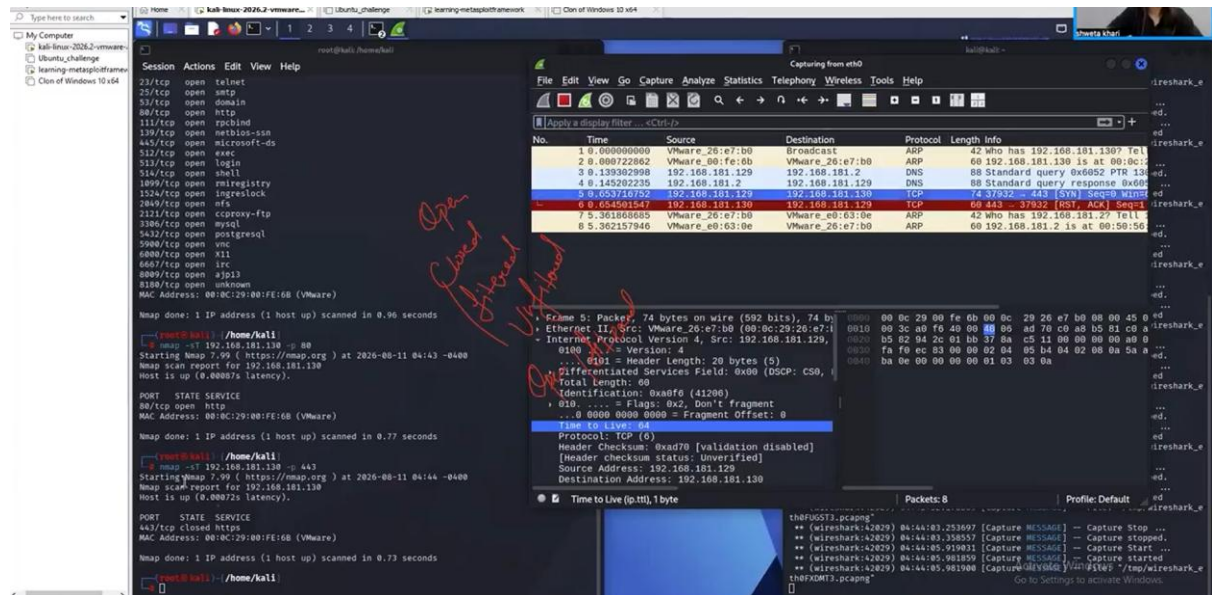
The port can be open, closed, filtered, undetermined (maybe firewall blocking), open filtered (opened or filtered) , close-filtered (Either closed or filtered)

Unfiltered – req is blocked by FW



Scanning for 443

Output: wireshark o/p shows RST, so the port is closed



NOTE:

Any scanning activity, results in RST Flag means that port is closed

FIN scan:

\$nmap -sF <IP> -p 80

Nmap can't determine the port is either open or filtered

[illegible]

```

root@kali: ~/home/kali
# nmap -sH 192.168.181.130 -p 443
Starting Nmap 7.99 (https://nmap.org) at 2026-08-11 04:49 -0400
Nmap scan report for 192.168.181.130
Host is up (0.00075 latency).

PORT STATE SERVICE
443/tcp closed http
MAC Address: 00:0C:29:00:FE:6B (Vmware)

Nmap done: 1 IP address (1 host up) scanned in 0.74 seconds

root@kali: ~/home/kali
# nmap -sH 192.168.181.130 -p 80
Starting Nmap 7.99 (https://nmap.org) at 2026-08-11 04:50 -0400
Nmap scan report for 192.168.181.130
Host is up (0.00058 latency).

PORT STATE SERVICE
80/tcp closed http
MAC Address: 00:0C:29:00:FE:6B (Vmware)

Nmap done: 1 IP address (1 host up) scanned in 0.90 seconds

root@kali: ~/home/kali
# nmap -sH 192.168.181.130 -p 21
Starting Nmap 7.99 (https://nmap.org) at 2026-08-11 04:50 -0400
Nmap scan report for 192.168.181.130
Host is up (0.00066 latency).

PORT STATE SERVICE
21/tcp unfiltered ftp
MAC Address: 00:0C:29:00:FE:6B (Vmware)

Nmap done: 1 IP address (1 host up) scanned in 0.85 seconds

root@kali: ~/home/kali
# nmap -sU 192.168.181.130 -p 80
Starting Nmap 7.99 (https://nmap.org) at 2026-08-11 04:51 -0400
Nmap scan report for 192.168.181.130
Host is up (0.00075 latency).

```

No.	Time	Source	Destination	Protocol	Length	Info
45	156.989/43319	192.168.181.129	192.168.181.2	DNS	88	Standard query 0x89e PTR 13
46	156.994/38287	192.168.181.2	192.168.181.129	DNS	88	Standard query response 0x89e
47	151.534/58138	192.168.181.129	192.168.181.130	TCP	54	55709 → 21 [ACK] Seq=1 Ack=1
48	151.535/58034	192.168.181.130	192.168.181.129	TCP	60	21 → 55709 [RST] Seq=1 Win=0
49	217.607/801751	Vmware.26:e7:b0	Broadcast	ARP	42	who has 192.168.181.130? tel
50	217.608/55914	Vmware.26:e7:b0	Vmware.26:e7:b0	ARP	60	192.168.181.130 is at 00:0c:
51	217.735/74128	192.168.181.129	192.168.181.2	DNS	88	Standard query 0x53f PTR 13
52	217.742/32458	192.168.181.2	192.168.181.129	DNS	88	Standard query response 0x53f
53	218.418/22440	192.168.181.129	192.168.181.130	UDP	56	33768 → 80 Len=4
54	218.418/22440	192.168.181.130	192.168.181.129	ICMP	60	Destination unreachable (Port
55	222.941/184651	Vmware.26:e7:b0	Vmware.26:e7:b0	ARP	42	who has 192.168.181.2? Tell
56	222.943/12129	Vmware.eb:0c:8e	Vmware.26:e7:b0	ARP	60	192.168.181.2 is at 00:50:50
57	228.186/55944	Vmware.26:e7:b0	Vmware.26:e7:b0	ARP	60	who has 192.168.181.129? Tel
58	228.186/57425	Vmware.26:e7:b0	Vmware.26:e7:b0	ARP	42	192.168.181.129 is at 00:0c:

```

Frame 53: Packet, 56 bytes on wire (448 bits), 56 captured (448 bits) on 0
Ethernet II, Src: Vmware.26:e7:b0 (00:0c:29:26:e7:b0), Dst: 00:00:00:00:00:00
Internet Protocol Version 4, Src: 192.168.181.129, Dst: 192.168.181.130
0100 ... = Version: 4
... .. = Header Length: 20 bytes (5)
... .. = Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-Set)
Total Length: 42
Identification: 0x6c41 (27713)
... .. = Flags: 0x0
... .. = 0x0000 0000 0000 = Fragment Offset: 0
Time to Live: 65
Protocol: UDP (17)
Header Checksum: 0xb2d0 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.181.129
Destination Address: 192.168.181.130

```

Time to Live (ttl), 1 byte

Packets: 58

Profile: Default

th0KpJRT3.pcang

== (Wireshark:42029) 04:48:05.228842 [Capture MESSAGE] -- Capture Stop ==

== (Wireshark:42029) 04:48:05.354868 [Capture MESSAGE] -- Capture stopped. ==

== (Wireshark:42029) 04:48:07.421149 [Capture MESSAGE] -- Capture Start ==

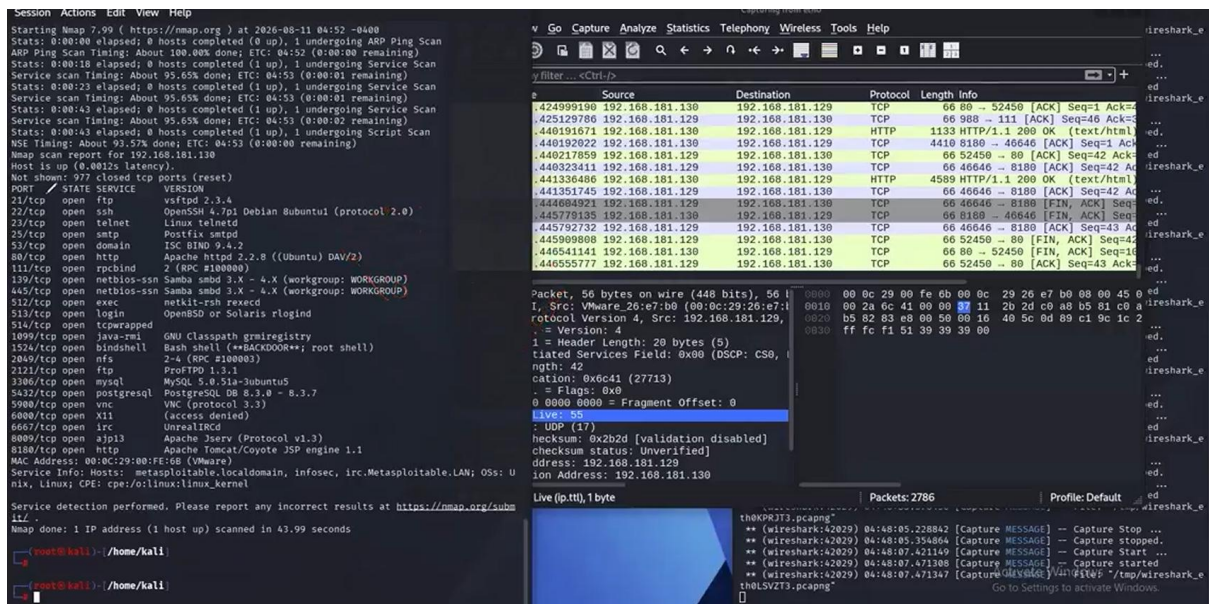
== (Wireshark:42029) 04:48:07.471388 [Capture MESSAGE] -- Capture started ==

== (Wireshark:42029) 04:48:07.471347 [Capture MESSAGE] -- Capture started ==

th0LSVZT3.pcang"

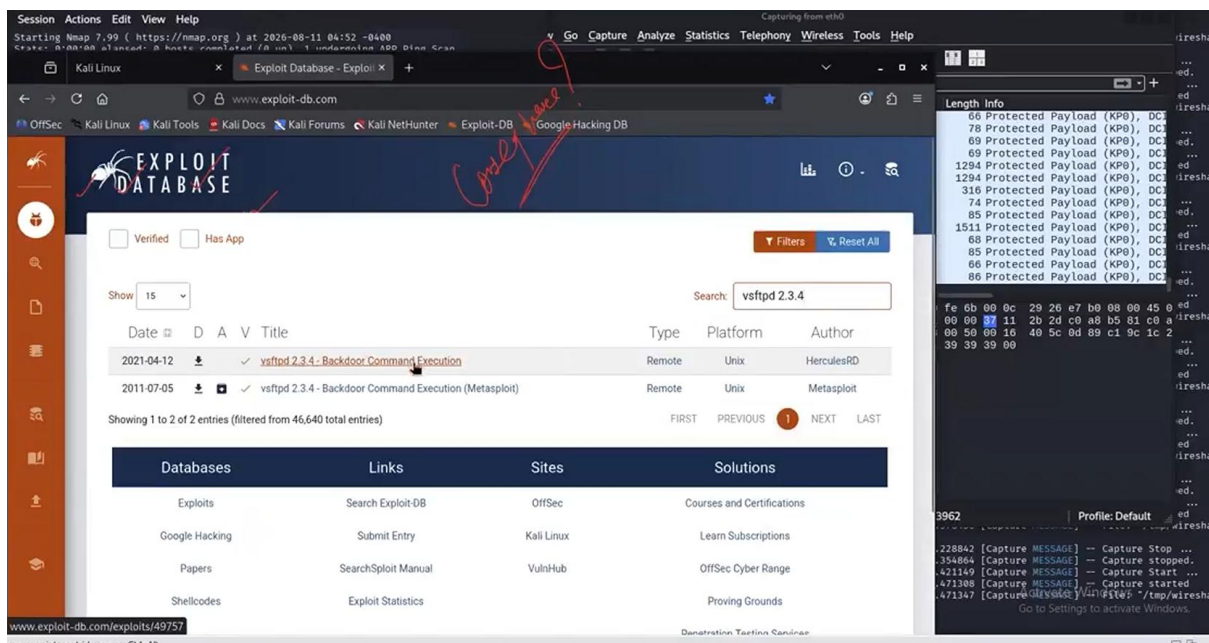
Go to Settings to activate Windows

```
$nmap -sV <IP>
```



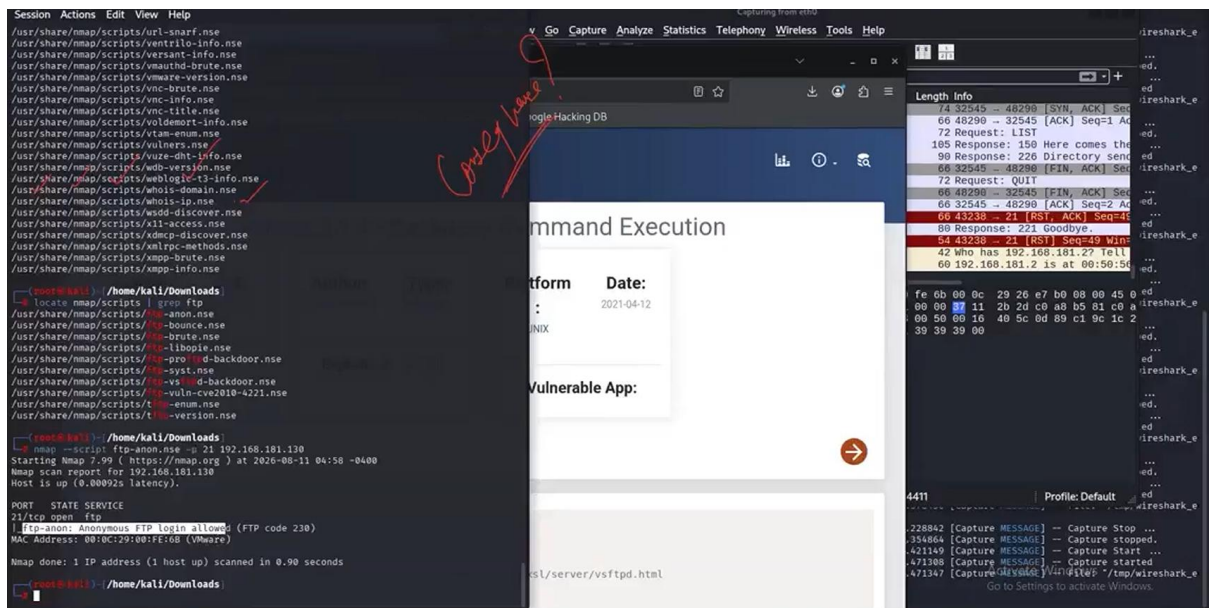
E.g FTP service version is available. You can refer know vulnerabilities from exploit-db.com website for this specific version and make exploitation.

If any of the target



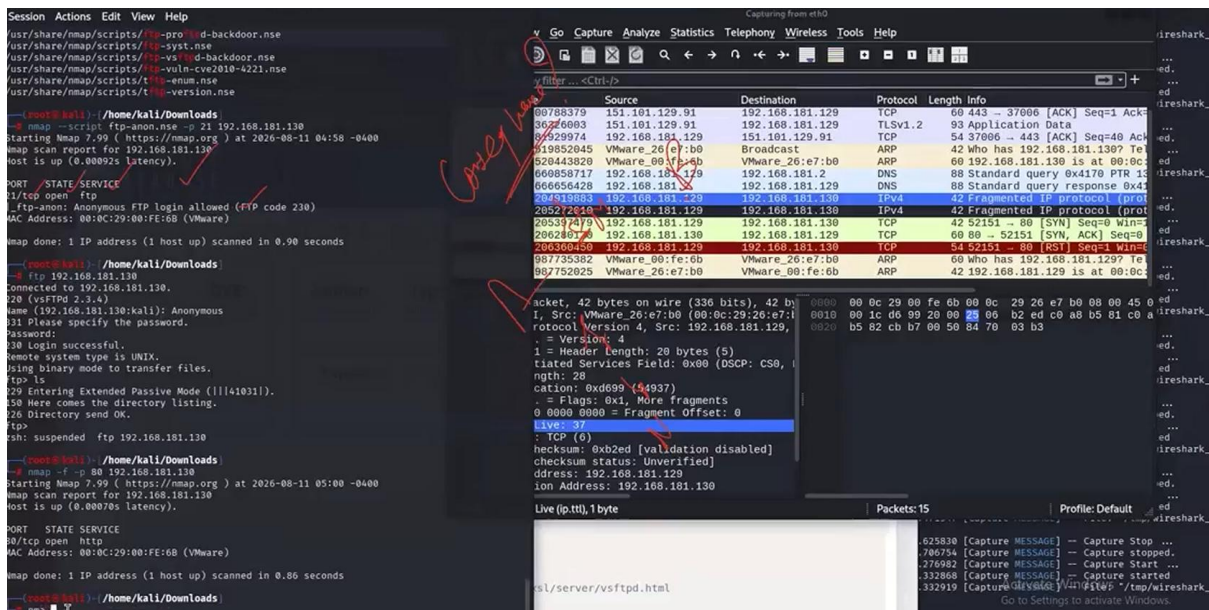
Download these exploits, and execute them in the shell using python3 command.

It provide a way to get access to ftp server and can execute these commands.



Fragmented Packet communication from Client to Server :

Use case: Say a server does not accept or allow syn packet, this can be sent to it through fragmented method such as sending syn packet as 's', 'y', 'n' as a separate fragments.



Time delayed packet sending:

```
Session Actions Edit View Help
ftp-anon: Anonymous ftp login allowed (FTP code 230)
MAC Address: 00:0C:29:00:FE:08 (VMware)
```

nmap done: 1 IP address (1 host up) scanned in 0.90 seconds

```
[root@kali:~]# nmap -sV 192.168.181.130
Nmap scan report for 192.168.181.130
220 (vsftpd 2.3.4)
Name [192.168.181.130]: Anonymous
301 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
220 Entering Extended Passive Mode (|||L4031)|.
150 Here comes the directory listing.
226 Directory send OK.
ftp>
```

zsh: suspended ftp 192.168.181.130

```
[root@kali:~]# nmap -sV 192.168.181.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-11 05:00 -0400
Nmap scan report for 192.168.181.130
Host is up (0.0007s latency).
```

PORT STATE SERVICE
80/tcp open http
MAC Address: 00:0C:29:00:FE:08 (VMware)

nmap done: 1 IP address (1 host up) scanned in 0.86 seconds

```
[root@kali:~]# nmap -sV 192.168.181.130 -p 80
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-11 05:01 -0400
Failed to resolve "-".
Note: --did you put a space between '-'?
Failed to resolve "p".
```

zsh: suspended nmap -T 0 192.168.181.130 -p 80

```
[root@kali:~]# nmap -sV 192.168.181.130 -p 80
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-11 05:01 -0400
Failed to resolve "-".
Note: --did you put a space between '-'?
Failed to resolve "p".
```

zsh: suspended nmap -T 0 192.168.181.130 -p 80

```
[root@kali:~]# nmap -sV 192.168.181.130 -p 80
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-11 05:01 -0400
Failed to resolve "-".
Note: --did you put a space between '-'?
Failed to resolve "p".
```

What Are TCP Communication Flags? A Guide to TCP Header Flags (SYN, ACK, FIN, RST, PSH, URG)

TCP communication flags are essential parts of the TCP header that govern the lifecycle of a TCP connection—from initiation using SYN, acknowledgment with ACK, graceful termination using FIN, abrupt termination using RST, to special instructions via PSH and URG. Each of these 1-bit control flags plays a critical role in managing data transmission and session control between hosts. These flags are not just important for developers and network engineers, but also for ethical hackers, penetration testers, and cybersecurity analysts who analyze packet behavior and detect anomalies using tools like Wireshark and Nmap. In this blog, we've explored the structure, function, and real-time use cases of TCP flags along with a breakdown of the TCP header format and diagrams for better understanding. Here are 30 frequently asked questions to clear all your doubts about TCP flags.

Quick answer: TCP flags are single-bit control fields in the TCP header that manage a connection's life cycle. The six you need to know are SYN (start a connection), ACK (confirm received data), FIN (close politely), RST (abort at once), PSH (deliver buffered data now) and URG (mark data as urgent). More than one flag can be set in the same segment, which is why a handshake reply shows SYN and ACK together.

Key takeaways

- Memorise the handshake as SYN, SYN-ACK, ACK, and recognise FIN for graceful close and RST for abrupt reset.
- Combinations such as SYN with FIN, or no flags at all, are abnormal and often point to scanning or crafted packets.
- Practise reading flags in a Wireshark capture or tcpdump, since CCNA, Security+ and SOC interviews all expect this.

Every reliable connection on the internet, from loading a web page to an SSH session, is steered by these flag bits. Reading them is a core skill for network engineers, SOC

analysts and anyone preparing for CCNA or Security+, because packet captures and firewall logs are full of them.

What are TCP flags?

TCP flags are control bits in the TCP header that tell each side what a segment is for. The modern TCP header carries a set of control bits; the six listed below handle the everyday work of opening, running and closing a connection. Each bit is either 1 (set) or 0 (not set), and several can be set at once.

Two older bits, CWR and ECE, support Explicit Congestion Notification. A further bit in the same byte, historically "reserved", was assigned to the experimental ECN nonce (NS) and later returned to reserved. You rarely touch these in day-to-day work, so this guide focuses on the six control flags.

What do the six main TCP flags do?

Each flag signals one kind of event. The table gives the short version; the notes after it add the detail that matters in practice.

Flag	Name	Signals	Typical moment
SYN	Synchronise	Start a connection and agree initial sequence numbers	First packet of the handshake
ACK	Acknowledgement	The acknowledgement number is valid; data received	Almost every packet after the first SYN
FIN	Finish	This side has no more data and wants to close	Graceful shutdown
RST	Reset	Abort the connection immediately	Packet to a closed port, or an unexpected state
PSH	Push	Pass buffered data to the application now	Interactive traffic such as SSH keystrokes
URG	Urgent	The urgent pointer field marks priority data	Rare; some legacy Telnet control signals

- **SYN** only appears when a connection is being set up. A second SYN on an established connection is abnormal and often flagged by security tools.
- **ACK** is set on nearly all segments once the handshake is done, because TCP acknowledges data continuously. A packet with ACK but no SYN or FIN is usually ordinary data or a plain acknowledgement.

- Where do the flags sit in the TCP header?

0 1 2 3

0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1

| Source Port | Destination Port |

| Sequence Number |

| Acknowledgement Number |

| Data | Reserved | U | A | P | R | S | F | Window Size |

| Offset| |R|C|S|S|Y|I| |

| | | G | K | H | T | N | N | |

| Checksum | Urgent Pointer |

| Options and Padding |

$$+ \text{-----} +$$

Each flag is one bit, so the whole set fits in part of a single byte. When you read a capture, the tool decodes these bits for you, but knowing the position helps when you write filters by hand.

How do flags drive the 3-way handshake?

Opening a TCP connection takes three segments, each defined by its flags:

1. **SYN:** the client sends a segment with SYN set and its initial sequence number.
2. **SYN-ACK:** the server replies with both SYN and ACK set, acknowledging the client and sending its own sequence number. This is the clearest example of two flags in one segment.
3. **ACK:** the client acknowledges the server's SYN. The connection is now established and data can flow.

Closing is the mirror image. One side sends FIN, the other acknowledges with ACK, then sends its own FIN, which the first side acknowledges. If anything goes wrong in between, either side can send RST to tear the connection down at once. If you want the layer-by-layer context behind this, see our explainer on [the difference between IP, TCP and UDP](#).

Which flag combinations are normal, and which are not?

Recognising unusual combinations is where this knowledge earns its keep.

Flags set	Meaning	Normal?
SYN	Connection request	Yes, first packet only
SYN, ACK	Connection accepted	Yes, handshake reply
ACK	Data or acknowledgement	Yes, most packets
PSH, ACK	Deliver this data now	Yes, interactive traffic
FIN, ACK	Close request with acknowledgement	Yes, end of session
RST or RST, ACK	Connection refused or aborted	Yes, but worth noticing
No flags set (NULL)	Empty control field	No, abnormal
FIN, PSH, URG together	Odd mix with no handshake	No, abnormal
SYN, FIN together	Contradictory (open and close)	No, abnormal

The abnormal rows are not just trivia. Older port-scanning techniques deliberately craft packets with no flags, or with FIN/PSH/URG together, to probe how a host responds. A

healthy network should rarely see them, so intrusion-detection rules often alert on exactly these patterns.

How do you see TCP flags yourself?

The fastest way to understand flags is to watch them. Capture a little traffic and filter on the flags.

In **tcpdump**, match SYN-only packets (new connection attempts):

```
sudo tcpdump 'tcp[tcpflags] == tcp-syn' -n -i any
```

Match SYN or ACK packets, the bulk of a handshake:

```
sudo tcpdump 'tcp[tcpflags] & (tcp-syn|tcp-ack) != 0' -n
```

In **Wireshark**, use display filters instead:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0 # new connection requests
```

```
tcp.flags.reset == 1 # resets
```

```
tcp.flags == 0x012 # exactly SYN+ACK
```

Load a capture, follow a single connection, and you will see the SYN, SYN-ACK, ACK sequence at the start and the FIN or RST at the end. Our guides to [Wireshark for beginners](#) and [Nmap for network scanning](#) show how to generate and read this traffic safely in a lab.

Why do TCP flags matter in security?

Flags are central to both attacks and defences, which is why they appear throughout the CCNA and Security+ syllabuses.

- **SYN floods:** an attacker sends many SYN packets and never completes the handshake, filling the server's half-open connection table. SYN cookies and rate limiting defend against this.
- **Scan detection:** SYN scans, FIN scans, NULL scans and the XMAS scan each use a distinct flag pattern. A defender who knows the normal set can spot the odd ones in logs.
- **Firewall behaviour:** stateful firewalls track which flags are valid for a connection's current state and drop segments that do not fit, such as an ACK for a connection they never saw open.
- **RST injection:** a forged RST can cut a connection. Seeing resets that do not match the traffic pattern is a sign worth investigating.

The right level of detail here is recognition and defence, not attack recipes. Study these patterns so you can read captures and tune detection rules.

Common misunderstandings

- **"Every packet has only one flag."** No. ACK rides along with most flags, and the handshake reply carries SYN and ACK together.
- **"PSH means the data is important."** PSH is about timing, not priority. It tells TCP to deliver buffered data now rather than wait.
- **"RST means an error occurred."** Not always. A RST to a closed port is the normal, correct response.
- **"URG is widely used."** It is effectively obsolete and often stripped by middleboxes.
- **"Flags live in the IP header."** The IP header has its own, different flags (such as Don't Fragment). TCP flags are in the TCP header.

Next step

Open a packet capture tool, start a single web or SSH connection, and filter for SYN, then RST, then FIN. Matching each flag to what you see on screen fixes the idea far better than re-reading a table. To take networking fundamentals through to certification, look at structured [CCNA networking training](#), where handshakes, flags and captures are practised hands-on.

Related reading

- [What Are TCP Communication Flags? | Understanding TCP Flags in Networking & Scanning](#)
- [What Really Happens When You Type a URL in Your Browser? Step-by-Step Guide Explained](#)
- [\[2026\] CCNA Voice Interview Questions](#)